Plan de tests

Projet « Testez et améliorez une application existante »

Ce document liste les tests que je vais écrire pour le back-end (Spring Boot) et le front-end (Angular). J'ai suivi la pyramide des tests : d'abord beaucoup de tests unitaires, qui sont rapides et simples, ensuite des tests d'intégration, et enfin quelques tests E2E qui vérifient l'application entière.

1. Répartition des tests

Type de test	Outils	Nombre de cas
Tests unitaires	JUnit 5 et Mockito (back), Jest (front)	23
Tests d'intégration	MockMvc et Testcontainers (back), Jest et TestBed (front)	14
Tests E2E	Cypress	4

Je vais commencer par les tests les plus simples : les services du back avec Mockito, puis les services du front. Ensuite, je ferai les tests d'intégration, et je finirai par les tests E2E.

2. Back-end : tests unitaires

Je m'appuie sur UserServiceTest, qui existe déjà : la classe testée est isolée, et ses dépendances (repository, encodeur de mot de passe, JwtService) sont remplacées par des mocks.

Classe	Ce que je teste	Entrée	Résultat attendu
StudentService	Créer un étudiant	Alice, Martin, alice@mail.com	l'étudiant est enregistré et renvoyé avec un id
StudentService	Lister les étudiants	2 étudiants en base	une liste de 2 étudiants
StudentService	Trouver un étudiant	id 1	l'étudiant 1
StudentService	Modifier un étudiant	id 1, nom Durand	l'étudiant avec le nom Durand
StudentService	Supprimer un étudiant	id 1	la suppression est appelée sur l'étudiant 1
UserService	Se connecter	login et bon mot de passe	un token
JwtService	Générer un token	utilisateur jdoe	un token qui contient jdoe
JwtService	Vérifier un token	token de jdoe	token valide
StudentDtoMapper	DTO vers entité	StudentRequestDTO	Student avec les mêmes infos
StudentDtoMapper	Entité vers DTO	Student	StudentResponseDTO avec les mêmes infos
CustomUserDetailService	Charger un utilisateur	login existant	l'utilisateur trouvé
JwtAuthenticationFilter	Requête avec un token	header Bearer + token valide	l'utilisateur est authentifié

3. Back-end : tests d'intégration

Je m'appuie sur UserControllerTest : l'application démarre avec une base MySQL dans Docker (Testcontainers), et j'envoie de vraies requêtes avec MockMvc. Pour les routes des étudiants, je me connecte avant chaque test pour récupérer un token.

Route	Entrée	Résultat attendu
POST /api/login	login et bon mot de passe	200 + token
POST /api/students	token + Alice, Martin, alice@mail.com	201 + étudiant créé avec un id
GET /api/students	token	200 + liste des étudiants
GET /api/students/{id}	token + id existant	200 + l'étudiant
PUT /api/students/{id}	token + id existant + nouveau nom	200 + étudiant modifié
DELETE /api/students/{id}	token + id existant	204
GET /api/students	sans token	401 (la route est bien protégée)

4. Front-end : tests unitaires

Je teste les services, le guard et l'intercepteur sans afficher d'écran. Pour les services HTTP, je vérifie que la bonne méthode est appelée sur la bonne URL.

Classe	Ce que je teste	Entrée	Résultat attendu
AuthService	Être connecté	token valide enregistré	isLoggedIn() renvoie true
AuthService	Se déconnecter	logout()	le token est supprimé
authGuard	Utilisateur connecté	isLoggedIn() = true	accès à la page
authGuard	Utilisateur non connecté	isLoggedIn() = false	redirection vers /login
authInterceptor	Ajouter le token	appel à /api/students avec un token	header Authorization: Bearer ajouté
UserService	Connexion	login et mot de passe	POST /api/login
StudentService	Lister	getAll()	GET /api/students
StudentService	Détail	getById(1)	GET /api/students/1
StudentService	Créer	create(étudiant)	POST /api/students
StudentService	Modifier	update(1, étudiant)	PUT /api/students/1
StudentService	Supprimer	delete(1)	DELETE /api/students/1

5. Front-end : tests d'intégration

Je teste chaque composant avec son template. Le service est remplacé par un mock, comme UserMockService dans le projet de départ.

Composant	Ce que je teste	Entrée	Résultat attendu
LoginComponent	Connexion	formulaire rempli	token enregistré, redirection vers /students
RegisterComponent	Inscription	formulaire rempli	redirection vers /login
StudentListComponent	Affichage de la liste	2 étudiants	2 lignes dans le tableau
StudentListComponent	Suppression	clic sur Delete	la ligne disparaît
StudentDetailComponent	Affichage du détail	/students/1	les infos de l'étudiant 1
StudentFormComponent	Ajout	/students/new, formulaire rempli	étudiant créé, redirection vers son détail
StudentFormComponent	Modification	/students/1/edit	champs pré-remplis, étudiant modifié

6. Tests E2E

Ces tests se lancent avec le back, le front et la base démarrés. Ils suivent le parcours d'un agent de la bibliothèque.

N°	Scénario	Ce que je fais	Ce que je dois voir
1	Page protégée	j'ouvre /students sans être connecté	je suis redirigé vers /login
2	Inscription et connexion	je crée un compte puis je me connecte	j'arrive sur la liste des étudiants
3	Gestion d'un étudiant	j'ajoute un étudiant, je le modifie, puis je le supprime	chaque action est visible à l'écran
4	Déconnexion	je clique sur Logout	je reviens sur /login et je ne peux plus accéder aux étudiants

7. Couverture de code

L'objectif est d'avoir au moins 80 % de couverture sur tout le code. Pour le back, j'utilise JaCoCo, qui génère un rapport avec la commande mvn clean verify. Pour le front, j'utilise la couverture de Jest (jest --coverage). Si je n'atteins pas 80 %, j'ajouterai des tests sur les cas d'erreur (mauvais mot de passe, étudiant introuvable, email déjà utilisé).